

StockKeep

Shop Inventory & Stock Tracking System

Software Requirements Specification

SRS.pdf — Standalone Requirements Document

Student Name:	Prince Ephraim Quarshie
Student ID:	22424601
Project Title:	StockKeep — AI Shop Inventory & Stock Tracking System
Institution:	University of Ghana
Department:	Computer Science
Module / Course:	CSCD602
Supervisor:	Prof. Solomon Mensah
Date:	13 August 2026

This document was produced by directly inspecting the StockKeep source code repository (Next.js 16 / React 19 / TypeScript / Prisma / SQLite). Every requirement below reflects functionality that exists in the codebase at the time of writing, referenced by file path where relevant. This is a standalone companion to *Project_Documentation.pdf* and must be read as consistent with it.

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Product Perspective	2
1.4	Stakeholders	2
1.5	User Classes and Characteristics	3
2	Functional Requirements	4
3	Non-Functional Requirements	9
4	System Interfaces	11
5	Database Requirements	12
6	Authentication Requirements	13
7	Security Requirements	14
8	Constraints	15
9	Assumptions	16
10	Acceptance Criteria	17
11	Requirements Prioritisation	18
12	Requirements Traceability	19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of **StockKeep**, a web-based shop inventory and stock tracking system. It is intended for the examining supervisor/marker, and for the student as a reference during any further development, testing, or maintenance of the system. This document was derived from direct inspection of the delivered source code rather than written speculatively, so that every requirement traces to an implemented (or explicitly flagged as absent) capability.

1.2 Scope

StockKeep allows a single small shop (one shared passcode, no per-user accounts) to:

- track inventory items, quantities, categories, suppliers, and reorder thresholds;
- record stock movements (stock-in and stock-out) and point-of-sale-style sales transactions;
- view a live dashboard, low-stock list, sales analytics, and inventory-value/dead-stock reports;
- manage supplier records;
- receive AI-generated (Google Gemini) descriptions, chat answers, stockout forecasts, and dashboard insights, each with a deterministic fallback if the AI service is unavailable;
- configure basic store settings.

Out of scope for the current version: multi-user accounts/roles, barcode scanning, multi-store/multi-tenant support, offline mode, and payment processing.

1.3 Product Perspective

StockKeep is a self-contained, full-stack Next.js 16 (App Router) web application. It is not a component of a larger system and has no dependency on other in-house systems. It integrates with two external third-party services: the Google Gemini API (`gemini-1.5-flash`, via `@google/generative-ai`) for AI features, and the Resend email API for low-stock notification emails. Data is persisted in a local SQLite database file (`prisma/dev.db`) accessed through the Prisma ORM.

1.4 Stakeholders

Stakeholder	Interest
Shop owner/manager	Accurate, real-time visibility of stock levels, sales, and reorder needs
Shop staff (end user)	A fast, simple interface to record stock-in/out and sales during a shift

Stakeholder	Interest
Student developer	Deliver a working, demonstrable, and academically documented system
Module supervisor/examiner	Assess the system and documentation against the marking rubric
Suppliers (indirect)	Represented as data records only; not system users

1.5 User Classes and Characteristics

As of the admin panel added in this revision, two distinct user classes now exist (earlier editions of this SRS stated there was only one; that is no longer accurate and is corrected here):

- **Shop Staff (Passcode User):** any person who knows the shared application passcode, entered at `/login`. Has full access to inventory, stock, sales, suppliers, reports, settings, and AI features, but no access to `/admin` or any `/api/admin/*` route, and is blocked from all write operations when an administrator enables User Lock Mode (FR-28).
- **Administrator:** any person who knows the separate admin passcode, entered at `/admin/login`. Has every Shop Staff permission (an admin session carries both the regular and admin cookies) plus exclusive access to the Admin Panel (`/admin`): system-wide KPIs, the full audit log, login/session history, toggling User Lock Mode, and changing the admin passcode. Administrators are never subject to User Lock Mode.

Both roles authenticate via a shared passcode model (one passcode per role, not per individual), so within a role there is still no per-user identity — the audit trail (FR-25) records the *role* that performed an action, not an individual person. See Section 6 (Authentication Requirements) for the mechanism.

A third, non-human actor is the **External AI Service (Google Gemini)**, which the system calls out to and receives responses from, with the system remaining fully functional (via fallback logic) if this actor is unavailable.

2. Functional Requirements

Each requirement has a unique ID used for traceability into `Testing_Report.pdf` (Section: Requirements-to-Test Traceability) and `Project_Documentation.pdf`.

ID	Requirement	Source (file)
FR-01	The system shall authenticate a user via a single shared passcode, hashed and checked against the <code>Settings.passcode</code> value (bootstrapped from an environment-configured default if unset).	<code>src/app/api/auth/login/route.ts</code> , <code>src/lib/auth.ts</code> , <code>src/lib/passcode.ts</code>
FR-02	The system shall set an HTTP-only session cookie (<code>stockkeep_auth</code>) on successful login, valid for 7 days.	<code>src/lib/auth.ts</code>
FR-03	The system shall block access to all pages and API routes except <code>/login</code> , <code>/api/auth/*</code> , and <code>/api/seed</code> until the session cookie is present.	<code>src/middleware.ts</code>
FR-04	The system shall allow an authenticated user to log out, clearing the session cookie.	<code>src/app/api/auth/logout/route.ts</code>
FR-05	The system shall display a dashboard with key inventory and sales KPIs, a 14-day revenue trend, category breakdown, and recent activity.	<code>src/app/api/analytics/route.ts</code> , <code>src/app/page.tsx</code>
FR-06	The system shall allow a user to add a new inventory item (name, category, quantity, threshold, price, location, description, expiry date, supplier), auto-generating a unique SKU.	<code>src/app/api/items/route.ts</code>

ID	Requirement	Source (file)
FR-07	The system shall validate new/updated item data (non-negative integers for quantity/threshold, non-negative price, required name) and reject invalid submissions with HTTP 400.	<code>src/lib/validations.ts</code> , <code>src/app/api/items/route.ts</code>
FR-08	The system shall allow a user to search and filter the inventory list by name/SKU, category, and low-stock status.	<code>src/app/api/items/route.ts</code>
FR-09	The system shall display a detail view for a single item, including its supplier, stock-movement history, and sales history.	<code>src/app/api/items/[id]/route.ts</code>
FR-10	The system shall allow a user to edit or delete an existing item.	<code>src/app/api/items/[id]/route.ts</code>
FR-11	The system shall allow a user to record a stock adjustment (Stock-In or Stock-Out) for an item, validating quantity as a positive integer.	<code>src/lib/validations.ts</code> (<code>StockAdjustmentSchema</code>), <code>src/app/api/items/[id]/adjust/route.ts</code>
FR-12	The system shall reject a Stock-Out adjustment that would reduce an item's quantity below zero.	<code>src/app/api/items/[id]/adjust/route.ts</code>
FR-13	The system shall apply a stock adjustment as a single atomic database transaction (update <code>Item.quantity</code> and create a <code>StockMovement</code> record together).	<code>src/app/api/items/[id]/adjust/route.ts</code>
FR-14	The system shall attempt to send a low-stock notification email (best-effort, failure does not block the request) when an item's quantity falls to or below its threshold after an adjustment.	<code>src/app/api/items/[id]/adjust/route.ts</code> , <code>src/lib/notify.ts</code>

ID	Requirement	Source (file)
FR-15	The system shall allow a user to record a sale (item, quantity, unit price), automatically decrementing stock and creating a matching StockMovement , rejecting sales that exceed available stock.	<code>src/app/api/sales/route.ts</code>
FR-16	The system shall allow a user to create, update, and delete supplier records (name, contact, phone, email, address).	<code>src/app/api/suppliers/route.ts</code> , <code>src/app/api/suppliers/[id]/route.ts</code>
FR-17	The system shall display an inventory summary report: total inventory value, category breakdown, and dead-stock detection.	<code>src/app/api/reports/summary/route.ts</code>
FR-18	The system shall allow a user to view and update store settings (store name, notification email toggle, notification email address, Resend API key, passcode field).	<code>src/app/api/settings/route.ts</code>
FR-19	The system shall provide an AI chat assistant that answers natural-language questions about current inventory/sales, using Gemini with a rule-based fallback reply if the AI call fails.	<code>src/app/api/ai/chat/route.ts</code>
FR-20	The system shall provide an AI-generated product description for an item, falling back to a templated description if the AI call fails.	<code>src/app/api/ai/describe/route.ts</code>
FR-21	The system shall provide a stockout forecast per item (estimated days until stockout, reorder quantity, urgency), computed deterministically from sales velocity and optionally enriched by Gemini; the deterministic result is always available even if Gemini fails.	<code>src/app/api/ai/forecast/[itemId]/route</code>

ID	Requirement	Source (file)
FR-22	The system shall provide AI-generated dashboard insights, cached in memory for 5 minutes, with a rule-based fallback if the AI call fails.	<code>src/app/api/ai/insights/route.ts</code>
FR-23	The system shall automatically populate the database with a representative dataset (4 suppliers, 12 items, 14 days of historical sales) the first time it detects an empty <code>Item</code> table, with no manual “generate demo data” step required and no destructive reset capability exposed anywhere.	<code>src/lib/seed.ts</code> , <code>src/app/api/seed/route.ts</code>
FR-24	The system shall authenticate an administrator via a separate admin passcode, hashed and checked independently from the regular user passcode, at a distinct <code>/admin/login</code> route, granting both the regular session cookie and a separate admin session cookie on success.	<code>src/app/api/auth/admin-login/route.ts</code> , <code>src/app/admin/login/page.tsx</code>
FR-25	The system shall record every state-changing action (item create/update/delete, stock adjustment, sale, supplier create/update/delete, settings update, lock-mode change, admin passcode change) to an audit log, capturing the action type, the role that performed it, an optional entity ID, and optional structured details.	<code>src/lib/audit.ts</code> , write routes under <code>src/app/api/**</code>
FR-26	The system shall record every login attempt (both roles, success and failure) with role, outcome, IP, and timestamp, viewable by an administrator.	<code>src/lib/audit.ts</code> (<code>logLoginEvent</code>), <code>src/app/api/admin/sessions/route.ts</code>

ID	Requirement	Source (file)
FR-27	The system shall provide an administrator-only dashboard showing system-wide KPIs (inventory, revenue, stock health, supplier count, audit log size, sessions today) and a live-refreshing recent-activity feed.	<code>src/app/api/admin/overview/route.ts</code> , <code>src/app/admin/page.tsx</code>
FR-28	The system shall allow an administrator to toggle a system-wide “User Lock Mode” that rejects write requests (item, stock, sale, supplier) from non-admin sessions with HTTP 403, while never restricting admin sessions.	<code>src/app/api/admin/lock/route.ts</code> , <code>src/lib/audit.ts</code> (<code>isUserReadOnly</code>)
FR-29	The system shall allow an administrator to change the admin passcode, requiring the current admin passcode to confirm the change, hashing the new value before storage.	<code>src/app/api/admin/settings/route.ts</code>

3. Non-Functional Requirements

ID	Category	Requirement	Status
NFR-01	Security	All state-changing and data-viewing routes (other than login/seed) must require a valid session cookie.	Implemented (<code>src/middleware.ts</code>)
NFR-02	Security	Passwords/passcodes should be hashed at rest and never compared or stored in plaintext.	Met - <code>src/lib/passcode.ts</code> hashes with <code>scrypt</code> ; verified live login rejects the old plaintext-era default and accepts a newly-hashed value (<code>Testing_Report.pdf</code> V-02, V-03). See Section 6.
NFR-03	Reliability	AI-dependent features must degrade gracefully (never return HTTP 500) if the external Gemini API is unreachable, rate-limited, or returns malformed data.	Implemented (<code>src/app/api/ai/*</code>)
NFR-04	Reliability	Stock-affecting writes (adjustment, sale) must be atomic - a failure partway through must not leave <code>Item.quantity</code> and <code>StockMovement</code> inconsistent.	Implemented via Prisma <code>\$transaction</code>
NFR-05	Data Integrity	Stock quantity must never be allowed to go negative through the adjustment endpoint.	Implemented (<code>items/[id]/adjust/route.ts</code>)

ID	Category	Requirement	Status
NFR-06	Maintainability	The codebase should be written in strictly-typed TypeScript to reduce a class of runtime errors.	Implemented (<code>tsconfig.json</code> , <code>"strict": true</code>)
NFR-07	Portability	The application must be deployable to Vercel's serverless Next.js runtime without manual server configuration.	Implemented, with a caveat - see NFR-10
NFR-08	Usability	The interface should be usable on both desktop and mobile viewport widths.	Styled with Tailwind CSS responsive utility classes; not verified across devices - execution required by student
NFR-09	Performance	Typical API responses should return within an acceptable interactive latency for a single-shop, low-concurrency workload.	Not benchmarked - execution required by student
NFR-10	Scalability	The persistence layer should tolerate the target deployment environment's execution model.	At risk - SQLite is a single-file, single-writer database, and Vercel serverless functions use an ephemeral filesystem; this is a known architectural constraint, not a defect introduced by this documentation. See <code>Technical_Debt_Plan.pdf</code> (TD-08).
NFR-11	Testability	Core business logic should be covered by automated tests.	Not met - zero automated test files exist in the repository at the time of writing. See <code>Testing_Report.pdf</code> .

4. System Interfaces

Interface	Description
Browser <-> Next.js server	HTTPS, JSON request/response bodies, cookie-based session, same-origin (no CORS configuration present)
Next.js server <-> SQLite	Prisma Client (<code>src/lib/prisma.ts</code>), parameterised queries only - no raw SQL found in the codebase
Next.js server <-> Google Gemini API	HTTPS, <code>@google/generative-ai</code> SDK, model <code>gemini-flash-latest</code> (a Google-maintained alias, chosen to resist future model retirements - see TD-13), authenticated via <code>GEMINI_API_KEY</code> environment variable (<code>src/lib/gemini.ts</code>)
Next.js server <-> Resend API	HTTPS, triggered from <code>src/lib/notify.ts</code> for low-stock alert emails, using a per-store API key stored in the <code>Settings</code> table

5. Database Requirements

The system requires a relational (or relational-compatible) data store with support for auto-incrementing integer primary keys, foreign keys with `onDelete` cascade/set-null behaviour, and `DateTime` columns with default/auto-update semantics. The current implementation uses **SQLite** via Prisma (`prisma/schema.prisma`) with seven models: `Item`, `StockMovement`, `Sale`, `Supplier`, `Settings`, and — added with the admin panel — `AuditLog` and `LoginEvent` (both independent append-only logs, no foreign keys to other tables). `Settings` gained two columns: `adminPasscode` (hashed, mirrors `passcode`) and `userReadOnly` (boolean, drives FR-28). See `Supporting_Files/ER_Diagram/er_diagram.png` (Figure D1) and `Project_Documentation.pdf` Section 17 for the full entity-relationship design.

6. Authentication Requirements

- AUTH-01: The system shall require a passcode to establish a session (FR-01).
- AUTH-02: The system shall persist the session as an HTTP-only, `SameSite=Lax` cookie, marked `Secure` when `NODE_ENV=production` (`src/lib/auth.ts`).
- AUTH-03: The system shall reject unauthenticated API requests with HTTP 401 and redirect unauthenticated page loads to `/login` (`src/middleware.ts`).
- AUTH-04 (*resolved*): The passcode configured through the Settings screen (`Settings.passcode` in the database) is the one checked at login. Login now hashes and reads `Settings.passcode` as the source of truth (`src/lib/passcode.ts`, `src/app/api/auth/login/route.ts`), falling back to a hashed `APP_PASSCODE` only to bootstrap the very first row. Was previously a gap (TD-01); verified fixed live in `Testing_Report.pdf` (V-02, V-03).
- AUTH-06 (*resolved*): Login attempts are rate-limited (`src/lib/rateLimit.ts`, 5 attempts / 5 minutes per client IP). Was previously a gap (SEC-05/TD-03); verified fixed live (`Testing_Report.pdf` V-10). Documented limitation: in-memory, per-process only.
- AUTH-05 (*superseded* — see AUTH-07/AUTH-08): Earlier editions of this SRS stated there was no role-based authorization. **This is no longer true.** An admin panel was added with a genuine two-tier role system; this entry is retained only so the change is traceable, and is fully replaced by AUTH-07 and AUTH-08 below.
- AUTH-07 (*new*): The system shall grant administrator privileges only via a separate admin passcode at `/admin/login`, independently hashed from the regular passcode (`Settings.adminPasscode`), issuing a distinct cookie (`stockkeep_admin`) that `src/middleware.ts` requires for every `/admin` page and `/api/admin/*` route, returning 403 (API) or redirecting to `/admin/login` (pages) otherwise. Regular authentication alone is insufficient for admin access. Verified live in `Testing_Report.pdf` (admin panel review pass).
- AUTH-08 (*new*): On first use, if no admin passcode has been explicitly set, the system shall hash and persist a configured default (`ADMIN_PASSCODE` environment variable, or a built-in fallback for local development only) rather than comparing the plaintext default on every request indefinitely — closing the exposure window to a single bootstrap write. This pattern was found to be violated in the initial admin-panel implementation (a live plaintext-comparison fallback with a default value that had been printed directly in `README.md`) and was fixed before that code was ever committed or deployed; see `Technical_Debt_Plan.pdf`, TD-16.

7. Security Requirements

- SEC-01: All Prisma database access shall use the typed query builder, not raw/unparameterised SQL (verified — no `$queryRawUnsafe/$executeRawUnsafe` usage found).
- SEC-02: Secrets (Gemini API key) shall be supplied via environment variables, not committed to source control (`.env*` is git-ignored).
- SEC-03 (*resolved*): The Resend API key is stored in the `Settings` database table but is **no longer** returned by `GET /api/settings` — the endpoint now returns a `resendApiKeyConfigured` boolean instead, and the Settings UI field is write-only. Was previously a gap; verified fixed live (`Testing_Report.pdf` V-04). See TD-06.
- SEC-04 (*resolved*): `POST /api/seed` now requires authentication like every other data route (removed from the middleware's public whitelist). Was previously reachable without authentication and could wipe all data via `force=true`. Verified fixed live: unauthenticated call now returns 401 (`Testing_Report.pdf` V-01). See TD-02.
- SEC-05 (*resolved*): Login attempts are now rate-limited (5 per 5 minutes per client IP). Was previously unlimited. Verified fixed live (`Testing_Report.pdf` V-10). See TD-03.
- SEC-06 (*gap*): No CSRF token is used on state-changing requests; reliance is on the `SameSite=Lax` cookie attribute alone. See TD-04.
- SEC-07: No security response headers (CSP, HSTS, X-Frame-Options, etc.) are configured in `next.config.ts`. See TD-05.

8. Constraints

- The system must run on the Next.js 16 App Router with React 19 and Node.js (Vercel-compatible runtime).
- The system must use Prisma as its ORM and SQLite as its bundled database provider (as configured in `prisma/schema.prisma`).
- AI functionality is constrained by third-party Gemini API availability, quota, and latency, which is why fallback logic is a hard requirement rather than an enhancement.
- The project was built under a short, fixed development window (consistent with the single-day initial commit followed by rapid iterative hardening visible in the Git history), which materially shaped several of the technical-debt items in `Technical_Debt_Plan.pdf`.

9. Assumptions

- The shop has a single point of sale / single physical location; no multi-branch requirement exists.
- All staff sharing the passcode are trusted equally; there is no requirement (yet) to distinguish or audit individual staff actions.
- Internet connectivity is available for AI and email features, but the system must remain usable for core inventory operations when it is not (satisfied by the fallback design).
- The examiner/marker will supply or generate their own live deployment, test accounts, and screenshots where this documentation marks them as “to be provided” or “not verified.”

10. Acceptance Criteria

A build is considered to satisfy this SRS when:

1. All Functional Requirements FR-01 through FR-29 can be exercised end-to-end from the UI without an uncaught server error.
2. No unauthenticated request (other than to `/login`, `/api/auth/*`) can read or modify data. **This is now met** — the former `/api/seed` exception (SEC-04) was fixed and verified live (`Testing_Report.pdf` V-01).
3. AI routes return a usable HTTP 200 response even with `GEMINI_API_KEY` unset or invalid (fallback path).
4. Stock quantities never go negative under any sequence of adjustment/sale requests.
5. The items outstanding in Section 12 (Non-Functional Requirements) marked “Not met” or “At risk” are acknowledged in the submission rather than claimed as complete.

11. Requirements Prioritisation

Using MoSCoW prioritisation:

Priority	Requirements
Must have	FR-01-FR-04 (auth), FR-06-FR-13 (core inventory CRUD + stock adjustment), FR-15 (sales), FR-24 (admin auth), NFR-01, NFR-04, NFR-05
Should have	FR-05 (dashboard), FR-16-FR-18 (suppliers, reports, settings), FR-25-FR-26 (audit/session log), FR-28 (lock mode), NFR-03 (AI graceful degradation), NFR-06
Could have	FR-19-FR-22 (AI chat/describe/forecast/insights), FR-27 (admin overview dashboard), FR-29 (admin passcode self-service) - valuable but not core inventory functionality
Won't have (this version)	Per-individual staff accounts (the two roles are still shared-passcode, not per-person identity), barcode scanning, multi-store support, automated test suite (explicitly deferred - see Technical_Debt_Plan.pdf)

12. Requirements Traceability

Full requirement-to-test mapping is maintained in `Testing_Report.pdf` (Requirements-to-Test Traceability table) and summarised in `Supporting_Files/Requirements_Traceability/requirements_traceab`. Every FR/NFR/AUTH/SEC ID defined in this document has a corresponding row in that matrix, cross-referenced to one or more Test IDs.